

```
In [ ]: from IPython.display import HTML
```

```
HTML("""
<style>

/* 1. Tamaño y espacio de los ítems de lista (li) */
.reveal ul li, .reveal ol li {
    font-size: 32px !important;    /* Ajusta este valor */
    margin-bottom: 15px !important; /* Espacio entre puntos */
    line-height: 1.3 !important;   /* Espacio entre líneas del mismo punto */
}

/* 2. Estilo de las sub-listas (listas dentro de listas) */
.reveal li > ul li, .reveal li > ol li {
    font-size: 0.85em !important; /* Un poco más pequeñas que las principales */
    list-style-type: circle !important;
}

/* 3. Cambiar el color de las viñetas (bullets) */
.reveal ul li::marker {
    color: #3498db !important;    /* Un azul sutil, cámbialo a tu gusto */
    font-weight: bold;
}

/* 4. Ajustar el ancho del contenedor para que las listas no se corten */
.reveal .slides {
    text-align: left !important;   /* Alinea el texto a la izquierda si prefieres */
}

/* 5. Tamaño y espacio de los párrafos (p) */
.reveal p {
    font-size: 30px !important;    /* Ajusta este valor */
    margin-bottom: 15px !important; /* Espacio entre puntos */
    line-height: 1.3 !important;   /* Espacio entre líneas del mismo punto */
}

/* 6. Tamaño y espacio de la celda de codigos view */
.reveal div pre {
    font-size: 20px !important;
}

/* 7. Tamaño y espacio de la celda de codigos ESTO ES EL CÓDIGO QUE SE MUESTRA EN UNA CELDA MD
```

```
.reveal div pre span {
    font-size: 32px !important;
}*/
/* Celdas de código edit*/
.reveal .cm-line {
    font-size: 24px !important;
}
/* 8. esTIL0 H1, H2 y H3*/
.reveal div h1{
    font-size: 48px !important;
    color: #4db6ac !important;
    text-transform: None;
}
.reveal div h2{
    font-size: 42px !important;
    text-transform: None;
}
.reveal div h3{
    /*font-size: 32px !important;*/
    text-transform: None !important;
}

</style>
""" )
```

Lenguaje Python

Cursada 2026



UNIVERSIDAD
NACIONAL
DE LA PLATA

Continuamos con el análisis de datos

- pandas
- NumPy
- Matplotlib
- Mostramos todo en Streamlit

⚡ Temas de la clase anterior

- Tipos de datos básicos de pandas:
- Series
- Dataframe

Conocer el dataset:

- primeras:
 - últimas:
 - filas y columnas:
 - columnas:
 - información de tipos de datos y cantidad de datos nulos:
 - cálculos básicos de estadística:
-
- primeras: **head()**
 - últimas: **tail()**
 - filas y columnas: **shape**
 - columnas: **columns**
 - información de tipos de datos y cantidad de datos nulos: **info()**
 - cálculos básicos de estadística: **describe()**

Acceso a los datos: posición numérica y por etiqueta

- posición numérica: **iloc**
- identificación por etiqueta: **loc**

Manipulación de Datos

- Filtrando los datos
- Modificación de nombres de columnas

```
In [ ]: import pandas as pd  
        from pathlib import Path
```

```
In [ ]: file_route = Path('ejemplos')/'clase10'
```

¿Qué pasa cuando no puede especificar claramente el tipo de datos?

- [Archivo utilizado.](#)
- [Descargado de la página de Indec.](#)
- Descripción: El programa nacional de producción permanente de indicadores sociales realiza encuestas trimestrales cuyo objetivo es conocer las características socioeconómicas de la población. Corresponde al 3er trimestre del 2024.

```
In [ ]: file_ind = 'usu_individual_T324.txt'
```

```
In [ ]: usi_ind = pd.read_csv(file_route/file_ind, delimiter=';')
```

Vemos el contenido de la columna antes de pasar la opción para que verifique mas detenidamente

```
In [ ]: usi_ind.iloc[:, 102]
```

Calculamos la cantidad de valores nulos de esa columna

```
In [ ]: nombres_columnas = usi_ind.columns  
nombre_columna_problematICA = nombres_columnas[102]  
usi_ind[nombre_columna_problematICA].isnull().sum()
```

- Se ven muchos valores nulos en la columna 'PP09A_ESP' por lo tanto no pudo detectar el tipo de dato.
- Leemos nuevamente pasando la opción 'low_memory=False'.

```
In [ ]: nusi_ind = pd.read_csv(file_route/file_ind, delimiter=';', low_memory=False)
```

```
In [ ]: nusi_ind.iloc[:, 102].head(10)
```

```
In [ ]: nombre_columna_problematICA = nombres_columnas[102]  
nusi_ind[nombre_columna_problematICA].isnull().sum()
```

Cantidad de filas:

```
In [ ]: nusi_ind.shape
```

- Del total de filas 47564, 47542 son nulas.
- Podemos ver el contenido de las filas que no son nulas.

```
In [ ]: nusi_ind[nusi_ind[nombre_columna_problematica].notnull()]['PP09A_ESP']
```



Reemplazos de valores en las celdas

- [Archivo utilizado.](#)
- Descargado del sitio de [ourairports](#).
- Descripción: Aeropuertos de Argentina por tipo y localidad.

```
In [ ]: file_route = Path('ejemplos')/'clase10'
file_data = 'ar-airports.csv'
df_airports = pd.read_csv(file_route/file_data)
```

```
In [ ]: df_airports.head(3)
```

Vemos el contenido de la columna 'region_name'

```
In [ ]: df_airports[df_airports.region_name.str.contains('Tierra del Fuego')]['region_name'].head(3)
```

¿Cómo reemplazamos?

```
In [ ]: df_airports.region_name.replace('Tierra del Fuego Province', 'Tierra del Fuego, Antártida e Islas del Atlántico Sur')
```

```
In [ ]: df_airports[df_airports.region_name.str.contains('Tierra del Fuego')]['region_name'].head(3)
```

¿Qué significa **inplace==True**?

Realicemos una modificación de texto

- 'Río Negro Province' -> 'Río Negro'

```
In [ ]: df_airports[df_airports.region_name.str.contains('Río Negro')]['region_name'].head(3)
```

- ¿Qué otra forma hay de filtrar, por ejemplo varios string?

```
In [ ]: df_airports.region_name.replace('Río Negro Province', 'Río Negro')
```

- ¿Se realizó el cambio?

```
In [ ]: df_airports[df_airports.region_name.str.contains('Negro')]['region_name'].head(3)
```

inplace hace el reemplazo sin tener que guardar el dataframe modificado en una nueva variable.



Numpy

- Librería de [Python](#) que fue optimizada para operaciones numéricas.
- Los elementos deben ser de un mismo tipo de datos.
- Contiene:
 - Arreglos multidimensionales definidos como *ndarray* que son rápidos y eficientes.
 - Funciones para realizar cálculos elemento por elemento con arreglos u operaciones matemáticas entre arreglos.
 - Herramientas para leer y escribir datasets a disco basados en arreglos.
 - Operaciones de álgebra lineal, transformada de Fourier y generación de números aleatorios.
 - Una API de C nativa que habilita extensiones de Python y a código C o C++ nativo para acceder a las estructuras de NumPy y facilidades computacionales.

- Las operaciones vectorizadas (arrays de NumPy) aplican una función a todos los elementos a la vez, usando código C optimizado y aprovechando la memoria contigua. Las listas de Python requieren bucles explícitos (comprensiones o for) que son más lentos.

Veamos un ejemplo de la eficiencia en las operaciones

```
In [ ]: import numpy as np
import random

lluvias = [random.randint(1, 100) for _ in range(10000)]
```

Convertimos la lista a un arreglo numpy

```
In [ ]: arr_lluvia = np.array(lluvias)
```

Algunas operaciones son similares al tipo **list** de Python

```
In [ ]: sum(arr_lluvia)
```

```
In [ ]: arr_lluvia[6]
```

```
In [ ]: arr_lluvia[5:20]
```

```
In [ ]: len(arr_lluvia)
```

```
In [ ]: arr_lluvia.shape
```


Desafío

Supongamos que con los datos numéricos en la variables **lluvias** queremos generar una nueva lista en la que cada valor sea el resultado de multiplicar por 5 el valor original correspondiente.

¿Cómo se puede realizar esta operación usando el tipo de dato **list** de Python?

```
In [ ]: %timeit [x * 5 for x in lluvias]
```

¿Cuál es el tiempo de esta operación usando el tipo de dato **array**?

```
In [ ]: %timeit arr_lluvia * 5
```

Operaciones aritméticas

- Numpy toma protagonismo cuando se tiene que realizar operaciones en "batch", es decir en lotes, simplificando el uso de **for**.
- Este procedimiento se denomina *vectorización*.

```
In [ ]: arr_lluvia + arr_lluvia
```

```
In [ ]: arr_lluvia - 5
```

 Comparemos sumar datos de una columna con array y con list

- Quiero sumar los valores de la columna 'CH03' del archivo de encuestas:

```
In [ ]: c_ch03 = nusi_ind.CH03
```

```
In [ ]: %timeit c_ch03.sum()
```

```
In [ ]: lista_columna = nusi_ind.CH03.to_list()
```

```
In [ ]: %timeit sum(lista_columna)
```

Más info sobre estos temas:

- Arrays multidimensionales.
- Tipos de datos para ndarrays.
- Indexación booleana: seleccionar elementos de un array utilizando un array de valores booleanos (True o False). Esta técnica permite filtrar datos de manera eficiente y concisa.
- Funciones aritméticas como sqrt, exp, etc.
- Ordenamiento.

En el capítulo 4.1 "Python for Data Analysis"



Reemplazar contenido

Renombrar una columna

- Por ejemplo cambiar el nombre de la identificación de una columna
- **'name' -> 'Name'**

Trabajemos con un dataframe más reducido.

```
In [ ]: df_reduced = df_airports[['type', 'name']]
```

```
In [ ]: df_reduced.rename(columns={'name': 'Name'}).head(3)
```

```
In [ ]: df_reduced.columns
```

Usamos **inplace** si estamos seguros de los cambios que vamos a realizar

```
In [ ]: df_reduced.rename(columns={'name': 'Name'}, inplace=True)
```

```
In [ ]: df_reduced
```

Esta operación genera **SettingWithCopyWarning**, ¿por qué? En detalle:

- [Documentación de pandas](#)
- [Nota con ejemplos](#)

SettingWithCopyWarning

- Algunas acciones en pandas retornan una vista de los datos y otras una copia

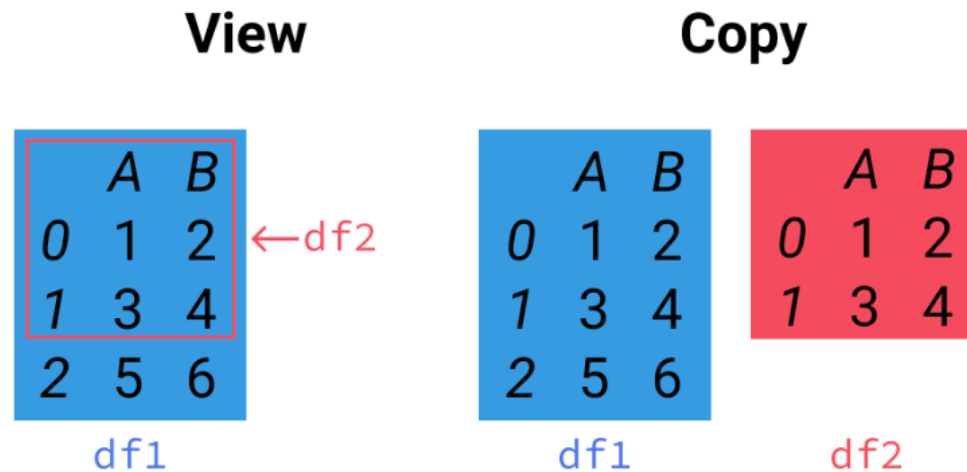


Imagen sacada de <https://www.dataquest.io/blog/settingwithcopywarning/>

SettingWithCopyWarning

- Por lo tanto, ante una modificación:

Modifying a View

	A	B
0	1	2
1	3	5
2	5	6

df1

Modifying a Copy

	A	B
0	1	2
1	3	4
2	5	6

df1

	A	B
0	1	2
1	3	5

df2

Imagen sacada de <https://www.dataquest.io/blog/settingwithcopywarning/>

Nos indica que pandas no puede asegurar que los resultados sean los esperados, ¿cómo podemos "asegurar" que no haya problemas?

Opción 1: La solución más recomendada

Combinar la selección y la asignación en una sola operación

```
In [ ]: df_reduced_copy = df_airports[['type', 'name']].rename(columns={'name': 'Nombre'})
df_reduced_copy.head(3)
```

¿Qué sucede en cada paso?

- `df_airports[['type', 'name']]` devuelve un nuevo DataFrame (generalmente una copia, no una vista).
- `.rename(columns={'name': 'Nombre'})` toma ese DataFrame y devuelve otro nuevo DataFrame con el nombre cambiado.
- Al final, `df_reduced_copy` recibe un objeto completamente independiente.

No hay modificación in-place (inplace=True), por lo que pandas no tiene que decidir si estás trabajando sobre una copia o el original.

Opción 2: Usar .copy() desde el principio

- se recomienda cuando realmente se necesita un subconjunto independiente (por ejemplo, porque se va a modificar varias veces).

```
In [ ]: df_reduced = df_airports[['type', 'name']].copy()  
df_reduced.rename(columns={'name': 'Nombre'}, inplace=True)
```

```
In [ ]: df_reduced
```

¿Y si queremos reemplazar varios valores de una columna?

```
In [ ]: reemplazo_tipos = {  
        'large_airport': 'Internacional',  
        'medium_airport': 'Regional',  
        'small_airport': 'Local'  
    }  
df_airports['type'].replace(reemplazo_tipos).unique()
```

```
In [ ]: df_airports['type'].unique()
```

Guardamos los reemplazos

```
In [ ]: df_airports['type'] = df_airports['type'].replace(reemplazo_tipos)
```

```
In [ ]: df_airports['type'].unique()
```



Modificación de tipo de datos

¿Es verdad que en 2024 se actualizaron mayor cantidad de datos de aeropuertos que en 2023 y menos que en 2021?

Veamos la columna **last_updated**

```
In [ ]: df_airports.last_updated.head()
```

- Contiene datos de fechas, pandas le asignó **object**, un tipo de datos general.
- Queremos verificar los años en que se realizaron actualizaciones.
 - ¿Con qué tipos de datos trabajamos en Python las fechas?

```
In [ ]: df_airports['last_updated'] = pd.to_datetime(df_airports.last_updated)
df_airports.last_updated.head(3)
```

Ahora que tenemos en el tipo de datos **datetime**: ¿cuáles son los años?

```
In [ ]: df_airports.last_updated.dt.year.unique()
```

```
In [ ]: df_airports[(df_airports.last_updated.dt.year==2023) ]
```

El atributo **.dt** en pandas se usa para acceder a las propiedades de los datos datetime, como:

- años
- meses
- días
- horas
- minutos, etc.
- En este caso, **.dt.year** se utiliza para obtener el año de cada fecha en la variable que contiene fechas.

¿Y los meses?

```
In [ ]: df_airports.last_updated.dt.month.unique()
```

En el ejemplo anterior:

```
df_airports['last_updated'] = pd.to_datetime(df_airports.last_updated)
```

Estamos trabajando con una variable **Series**, no con todo el dataframe.

Se modificó una columna con tipo de dato **object** a **datetime** de pandas usando:

- `pd.to_datetime(columna)`

¿Qué otras opciones tenemos para modificar tipos de datos y cómo lo hacemos?

Modificación de tipo de datos y eliminación de filas

- Deseamos trabajar con valores enteros que representen la elevación de cada aeropuerto.
- ¿Cómo cambiamos el tipo de datos con Python?
- Veamos la columna **elevation_ft**

```
In [ ]: df_airports.elevation_ft.head()
```

Para cambiar a tipos de datos como int, float ...usamos

```
df.column.astype(..: int, float, bool, str
```

Veamos el código y qué pasa:

```
In [ ]: df_airports.elevation_ft.astype(int)
```

- El problema es que tenemos valores nulos o infinitos, (Nan o Inf).
- Podemos asignarles un valor, por ejemplo 0, pero generará análisis erróneos cuando se involucre este valor.
- Podemos optar por:
 - eliminar las filas que tienen **non-finite values (NA or inf)**, ojo verificar que no perdemos información.

- en el caso que se necesite trabajar con estos valores, nos podemos quedar con un subconjunto del dataset sin las filas que contienen valores nulos.
- usar tipo nullable Int64 (con 'I' mayúscula): `df['col'].astype('Int64')` (pandas >= 0.24)

Veamos la opción de sacar las filas que contienen valores nulos:

```
df.dropna()
```

Nos permite eliminar estos casos del dataframe.

```
In [ ]: df_airports.dropna()
```

Quedaron sólo 3 filas!!!!



Eliminación de datos nulos

- En realidad no se eliminaron porque no modificamos de forma definitiva el Dataframe.

```
df.dropna()
```

- Elimina **todas** filas que contengan en **algunas** de las columnas valores nulos.

Vimos que algunas columnas tienen muchos datos faltantes como:

```
In [ ]: nulos_por_columna = df_airports.isnull().sum()

# Filtrar las columnas que tienen más de un valor nulo
nulos_por_columna[nulos_por_columna > 1]
```

Para eliminar las filas que contienen valores nulos **sólo de una columna**:

```
In [ ]: df_airports_mod = df_airports.dropna(subset=['elevation_ft']).copy()
```

Para no perder los datos originales, realizamos una **copia** sacando los valores nulos de la columna **elevation_ft**


```
In [ ]: df_airports_mod['elevation_ft'] = df_airports_mod.elevation_ft.astype(int)
df_airports_mod[['name', 'elevation_ft']].head(3)
```

¿Cuántos valores nulos hay en cada columna ahora?

```
In [ ]: df_airports_mod.info(memory_usage='deep')
```

Métodos para eliminar o completar datos:

- **dropna**: borra filas con datos nulos de Series o de Dataframe.
 - `df.dropna()`: elimina todas filas que contengan en algunas de las columnas valores nulos.
 - `df.dropna(subset=['columna'])`: elimina las filas que tiene valores nulos en esa columna.
- **fillna**: completar con valores las ocurrencias de datos faltantes.
 - `fillna(0)`: reemplaza todos las ocurrencias por un valor, en este caso 0.
 - `fillna({col1: 0.5, col2:0})`: completa datos faltantes con diferentes valores para diferentes columnas.
- **drop**: elimina filas o columnas
 - `df.drop(['col1', 'col2'], axis=1)` o `df.drop(columns=['col1', 'col2'])`: elimina las columnas col1 y col2.
 - `df.drop([0, 1])`: elimina las filas con índice 0 y 1.

Capítulo 5.2 "Python for Data Analysis"



Consigna para resolver en casa

- ¿Es verdad que hay mayor cantidad de aeropuertos `small_airport` que `medium_airport` que tienen elevación mayor que la media?
- ¿Es cierto que la elevación media de los aeropuertos actualizados en los últimos 5 años que hay registros es 921 aproximadamente?



Crear nuestro dataframe

```
In [ ]: recognized_women_info = {
    'Cecilia Berdichevsky': {'Area': 'Informática', 'Fecha_nacimiento': '1960-05-10'},
    'Noemí García': {'Area': 'Informática', 'Fecha_nacimiento': '1965-11-30'},
    'Cecilia Grierson': {'Area': 'Ciencia', 'Fecha_nacimiento': '1859-11-22'},
```

```
'Julieta Lanteri': {'Area': 'Ciencia', 'Fecha_nacimiento': '1873-11-22'},
'Jeanette Campbell': {'Area': 'Deporte', 'Fecha_nacimiento': '1965-03-20'},
'Mary Terán': {'Area': 'Deporte', 'Fecha_nacimiento': '1936-01-23'},
'Patricia Sosa': {'Area': 'Música', 'Fecha_nacimiento': '1956-01-23'},
'Fabiana Cantilo': {'Area': 'Música', 'Fecha_nacimiento': '1959-03-03'}
}
```

- Convertir el diccionario a DataFrame

```
In [ ]: women_df = pd.DataFrame.from_dict(recognized_women_info, orient='index')
```

orient=index: indica que las claves del diccionario deben ser interpretadas como índice del dataframe que se está pasando como parámetro

```
In [ ]: women_df
```

```
In [ ]: women_df.loc['Cecilia Berdichevsky':'Jeanette Campbell']
```

¿Con `iloc` cómo accedemos a las 2 primeras filas?

```
In [ ]: women_df.iloc[
```

- Convertir el índice en columna como parte del Dataframe

```
In [ ]: women_df.reset_index(inplace=True)
women_df.rename(columns={'index': 'Nombre'}, inplace=True)
women_df.index.name = 'index'
women_df
```

```
In [ ]: women_df.info()
```

Accedemos a filas por número de fila y por etiquetas

```
In [ ]: women_df.iloc[2]
```

```
In [ ]: women_df.loc[2]
```



Agregar columnas



Agregar una columna categorizada

Necesitamos analizar los aeropuertos por categorías. Agregaremos una columna **height** que contenga los valores:

- bajo
- medio
- alto
- muy alto

según los rangos de valores de la columna **elevation_ft**

Tenemos que **recorrer** el dataset y asignar el valor correspondiente, podemos hacerlo, recorriendo...usando **iterrows**

```
In [ ]: df_airports.elevation_ft.describe().loc['25%': '75%']
```

Hacemos una copia para probar varias formas de realizar lo mismo

```
In [ ]: df_airports_height = df_airports.copy()
heights = []
for index, row in df_airports_height.iterrows():
    value = row['elevation_ft']
    if value < 131.5:
        height = 'bajo'
    elif value < 314:
        height = 'medio'
    elif value < 900.5:
        height = 'alto'
    else:
        height = 'muy alto'
```

```
# Añadir el resultado a la lista
heights.append(height)
df_airports_height['height'] = heights
```

```
In [ ]: df_airports_height[['name', 'elevation_ft', 'height']].head(4)
```

Pero si hay algo que nos gusta de pandas es no tener que **recorrer** manualmente, entonces ¿cómo hacemos?

```
In [ ]: df_airports_height = df_airports.copy()
def define_height(value):
    if value < 131:
        return 'bajo'
    elif value < 314:
        return 'medio'
    elif value < 900.5:
        return 'alto'
    else:
        return 'muy alto'
```

```
In [ ]: df_airports_height['height'] = df_airports_height['elevation_ft'].apply(define_height)
df_airports_height[['name', 'elevation_ft', 'height']].head(4)
```

Apply de pandas permite aplicar una función a todas las filas de una columna:

- aplicar la función a cada fila o columnas de un Dataframe o a una Series.
- **Dataframe:**
 - **apply(func, axis=0):** axis=0, valor por defecto, aplica la función por cada columna, en este caso es una Series, por lo tanto solo a esa columna
 - **apply(func, axis=1):** axis = 1, aplica la función a cada fila.

De esta forma en lugar de usar **iterrows**, utilizamos **apply** con una función definida que genera el dato necesario.

Otras funciones que podemos aplicar:

- str.lower()
- map: se usa principalmente en Series.

- apply y map son similares: apply tiene más flexibilidad con el pasaje de parámetros
- Más info en:

Capítulo 7.2 "Python for Data Analysis"

```
In [ ]: df_airports_height['height'] = df_airports_height['elevation_ft'].map(define_height)
```

```
In [ ]: df_airports_height[['elevation_ft', 'height']].head(4)
```



Consigna para resolver en casa

Mejorar los cambios en el caso de valores **nan**

¿Qué pasó en estos casos?:

- ¿Está bien el valor correspondiente a la columna **'height'**?
- Investigar:
 - la función **pd.isna** para consultar si un valor es **nan**
 - la conversión al tipo de datos: **pd.Int64Dtype()**

```
In [ ]: df_airports_height[df_airports_height.elevation_ft.isnull()][['name', 'elevation_ft', 'height']].head(3)
```



Agregar columna en función de dos columnas del Dataframe

- Queremos categorizar en función del tipo y de la elevación.
- Se quiere definir la nueva columna en función de dos **columnas**

Hacemos una copia del dataframe y nos quedamos solamente con:

- 'large_airport'
- 'medium_airport'
- 'small_airport'

```
In [ ]: df_airports = pd.read_csv(file_route/file_data)
```

```
In [ ]: df_airports.type.unique()
```

```
In [ ]: columnas = ['large_airport', 'medium_airport', 'small_airport']  
df_airports_height = df_airports[df_airports.type.isin(columnas)]  
df_airports_height
```

```
In [ ]: df_airports_height = df_airports[df_airports.type.isin(columnas)].copy()  
def define_height(value):  
    match value:  
        case _ if value <= 131:  
            return "bajo"  
        case _ if value <= 314:  
            return "medio"  
        case _ if value <= 900.5:  
            return "alto"  
        case _:  
            return "muy_alto"  
def type_height(value, type_airport):  
    text_value = define_height(value)  
    match value:  
        case _ if type_airport == 'large_airport':  
            return 'large_'+text_value  
        case _ if type_airport == 'medium_airport':  
            return 'medium_'+text_value  
        case _:  
            return 'small_'+text_value
```

```
In [ ]: df_airports_height['height'] = df_airports_height.apply(lambda x: type_height(x.elevation_ft, x.type), axis=1)
```

```
In [ ]: df_airports_height[['name', 'type', 'elevation_ft', 'height']].head(3)
```

apply en este caso, no se especifica que se aplique a una sola columna porque necesitamos acceder a **dos columnas**.

```
df_airports_height.apply(lambda x: type_height(x.elevation_ft, x.type), axis=1)
```

- **axis=1**: indica que se aplique la función a cada fila

- **axis=0**, la opción por **defecto** lo hace sobre una columna



Nuestros datos y gráficos en Streamlit

- Vamos a trabajar con un archivo que cuenta con un registro de avistajes de felinos en nuestro país
- [Archivo utilizado](#).
- Descargado del sitio de [gbif](#) y adaptado para poder trabajar estos datos.

```
In [ ]: felinos = pd.read_csv(file_route/'felinos_filtrado.csv')
```

Veamos el contenido:

```
In [ ]: felinos.columns
```

```
In [ ]: felinos.head(3)
```

```
In [ ]: felinos.info()
```



¿Es verdad que se vieron felinos en todas las provincias de Argentina?

```
In [ ]: felinos.stateProvince.unique()
```

Arreglemos algunos problemas

```
In [ ]: felinos.stateProvince = felinos.stateProvince.replace('Neuquen', 'Neuquén')
```

```
In [ ]: felinos.stateProvince.unique()
```

Necesito saber, cuántos avistajes hay de cada tipo de felino (columna **genus**).



Agrupamientos

value_counts(): agrupamos, contamos y ordenamos

```
In [ ]: felinos.genus.value_counts(ascending=True)
```

value_counts(): devuelve una Series que contiene los conteos de valores únicos en orden **descendente**.

- **Uso:** Se utiliza comúnmente para analizar datos categóricos, permitiendo ver cuántas veces aparece cada categoría en los datos.
- Opciones:
 - **normalize:** si se establece en True, devuelve la proporción de cada valor en lugar del conteo.
 - **sort:** si se establece en True (por defecto), los resultados se ordenan en orden descendente.
 - **ascending:** si se establece en True, los resultados se ordenan en orden ascendente.
 - **dropna:** si se establece en True (por defecto), se excluyen los valores NaN del conteo.

¿Qué pasa con los valores nulos cuando contamos?

```
In [ ]: felinos.info()
```

```
In [ ]: #felinos.year = felinos.year.fillna(0)

#felinos.year = felinos.year.astype(int)
felinos.year.value_counts(dropna=False)
```

groupby(): agrupamos y contamos

```
In [ ]: felinos.groupby('genus').species.count()
```

Sigue tres pasos:

- Split (dividir): separa el DataFrame en grupos según la(s) columna(s).

- **Apply** (aplicar): ejecuta una función de agregación (count, sum, nunique, etc.) sobre cada grupo.
- **Combine** (combinar): reúne los resultados en un nuevo Dataframe/Series.
- **groupby()**: permite agrupar por una columna o más, pero necesita ir acompañado de alguna operación de agregación o transformación para poder visualizar el resultado correspondiente.
- **value_counts()**: similar con la opción mostrada de groupby, con la diferencia que ya lo muestra ordenado.

👉 Queremos saber qué felino (según species) se registró en el mayor número de provincias distintas.

```
In [ ]: felinos.groupby('species')['stateProvince'].unique()
```

```
In [ ]: felinos.groupby('species')['stateProvince'].nunique()
```

- **unique()** indica los **valores** únicos de la agrupación que hicimos en función de la columna que indicamos
- **nunique()** indica la **cantidad(n)** de valores únicos de esa columna, en este caso las provincias. Podemos visualizar las cantidades de cada columna, no sólo de stateProvince.

```
In [ ]: felinos.groupby('species').nunique()
```

👉 O podemos obtener qué cantidad de **diferentes tipos** de felinos se vieron en cada **provincia**.

```
In [ ]: felinos.groupby('stateProvince')['species'].nunique()
```

Métodos de agregación

Dijimos que **groupby** tiene que estar acompañado de alguna operación de agregación, algunas de las que se puede utilizar son:

- **sum()**: suma los valores de cada grupo.
- **count()**: cuenta el número de elementos en cada grupo.
- **mean()**: calcula la media de los valores en cada grupo.

Ejemplos usando **count()** , que cuenta la cantidad según los valores de columna **genus**.

```
In [ ]: felinos.groupby('genus')['kingdom'].count()
```

También podemos agrupar por dos columnas, y realizar la operación sobre una tercera columna que nos interesa.

Queremos saber la cantidad de localidades de cada provincia donde se vió cada tipo de felino

- agrupamos por **genus** primero.
- después por cada tipo de genus agrupa por **provincia**.
- y luego muestra la cantidad de valores únicos por localidad.
- Se calcula la cantidad de localidades diferentes de cada provincia que está en el Dataframe, donde se vieron los tipos diferentes de felinos.

```
In [ ]: felinos.groupby(['genus', 'stateProvince']).locality.nunique()
```

Operaciones anidadas

- Method Chaining (encadenamiento de métodos) y legibilidad
- pandas permite encadenar métodos
- Evita variables intermedias y hace el código más expresivo.
- Se usan paréntesis para separar la cadena en varias líneas.

1 Contar registros por género y renombrar la columna de cantidad

```
In [ ]: (felinos.groupby('genus')['kingdom']  
        .count()  
        .sort_values(ascending=False)  
        .reset_index(name='cantidad')  
        )
```

- `groupby('genus')` → agrupa por la columna `genus` (tipo de felino).
- `['kingdom']` → selecciona solo la columna `kingdom` (para contar sobre ella; elige cualquier columna no nula).

- `.count()` → cuenta cuántos registros hay en cada grupo (los valores NaN se ignoran).
- `.sort_values(ascending=False)` → ordena de mayor a menor cantidad.
- `.reset_index(name='cantidad')` → convierte el índice (genus) en una columna llamada genus y la columna de conteos se renombra a 'cantidad'.

```
In [ ]: felinos.columns
```

2 Mismo análisis, pero contando sobre la columna year

```
In [ ]: (felinos.groupby('genus')['year']
        .count()
        .sort_values(ascending=False)
        .reset_index(name='cantidad')
        )
```

- Es idéntico al anterior, pero la columna seleccionada es year en lugar de kingdom.
- `count()` sobre year cuenta los años no nulos por género (útil si hubiera NaN en year).

3 Agregaciones múltiples: contar y contar valores únicos en dos columnas

```
In [ ]: (felinos.groupby('genus')[['year', 'stateProvince']]
        .agg(['count', 'nunique'])
        .sort_values(['year', 'count'], ascending=False)
        )
```

- `groupby('genus')` → mismo agrupamiento.
- `[['year', 'stateProvince']]` → selecciona dos columnas para aplicar distintas funciones.
- `.agg(['count', 'nunique'])` → aplica dos funciones a cada columna seleccionada:
- `count`: número de valores no nulos.
- `nunique`: cantidad de valores distintos (únicos).

- `.sort_values(('year','count'), ascending=False)` → ordena el resultado por la columna que resulta de aplicar count a year (primero los géneros con más registros de año).

4 Filtrar grupos después de agrupar: solo provincias con más de 5 avistajes

```
In [ ]: (felinos.groupby('stateProvince')['genus']  
        .count()  
        .loc[lambda x: x > 5]  
        .sort_values(ascending=False)  
        )
```

- `groupby('stateProvince')['genus'].count()` → cuenta cuántos registros (avistajes) hay en cada provincia.
- `.loc[lambda x: x > 5]` → filtra las provincias que tienen más de 5 avistajes. La función lambda recibe la Serie de conteos y devuelve una máscara booleana.
- `.sort_values(ascending=False)` → ordena de mayor a menor cantidad.

Uso de los paréntesis en las expresiones

- Los paréntesis no son parte de la sintaxis de pandas, sino una característica de Python para escribir código más legible.
- Permiten distribuir la cadena de métodos en varias líneas sin errores de indentación.

Beneficios:

- Código más declarativo (se lee como una secuencia de transformaciones).
- Facilita el debugging (se puede comentar una línea).
- Reduce el riesgo de modificar accidentalmente objetos intermedios.



Graficamos

Vamos a ver algo más de Matplotlib



```
In [ ]: import matplotlib.pyplot as plt
```

Graficar datos desde Series

```
In [ ]: data = {  
        'Mes': ['Enero', 'Febrero', 'Marzo', 'Abril', 'Mayo'],  
        'Ventas': [200, 300, 250, 400, 350]  
    }  
df = pd.DataFrame(data)
```

- Crear un gráfico de líneas sin usar las etiquetas o índices del Dataframe:

```
In [ ]: df['Ventas'].plot(kind='line', marker='o')  
plt.show()
```

- El gráfico no muestra en el eje x las etiquetas y el gráfico no es claro

```
In [ ]: plt.figure(figsize=(10, 5))  
plt.plot(df['Mes'], df['Ventas'], marker='o')  
plt.title('Ventas Mensuales')  
plt.xlabel('Mes')  
plt.ylabel('Ventas')  
plt.grid()  
plt.show()
```

```
In [ ]: plt.figure(figsize=(8, 8))  
plt.pie(df['Ventas'], labels=df['Mes'], autopct='%1.1f%%', startangle=140)  
plt.title('Distribución de Categorías')  
plt.axis('equal') # Para que el gráfico sea un círculo  
plt.show()
```

Mostrar varios gráficos

```
In [ ]: df['Ventas'].plot(kind='line', marker='o')
#plt.pie(df['Ventas'], labels=df['Mes'], autopct='%1.1f%%', startangle=140)

plt.title('Ventas Mensuales')
plt.xlabel('Mes')
plt.ylabel('Ventas')
plt.xticks(rotation=0)
plt.show()
```

- Al generar varios gráficos en la misma figura, **Matplotlib** muestra solamente el último gráfico que generamos.

Grafiquemos las agrupaciones de datos hechas entre los tipos de felinos y las provincias.

```
In [ ]: species_unique_province = felinos.groupby('species')['stateProvince'].nunique()
province_unique_species = felinos.groupby('stateProvince')['species'].nunique().reset_index().sort_values(by='speci
paleta = plt.get_cmap('coolwarm')
```

```
In [ ]: # Configurar los datos para el gráfico de torta
labels_species = species_unique_province.index
sizes_species = species_unique_province.values
posicion_g = np.arange(len(labels_species))

# Configurar los datos para el gráfico de barra
labels_province = province_unique_species.stateProvince
sizes_province = province_unique_species.species

posicion = np.arange(len(labels_province))
# Crear subplots
fig, ax = plt.subplots(1, 2, figsize=(14, 6))
paleta = plt.get_cmap('coolwarm')
colors = [paleta(1. * i / len(labels_species)) for i in range(len(labels_species))]
ax[0].pie(sizes_species, labels=labels_species, colors=colors, autopct='%1.1f%%')
ax[0].set_title('Distribución de felinos por cantidad de provincias')
```

```
ax[1].bar(posicion , sizes_province, tick_label=labels_province)
ax[1].set_title('Cantidad de tipos de felinos diferentes vistos en cada provincia ')
ax[1].set_xticklabels(labels_province, rotation=45)

# Mostramos el gráfico
fig.tight_layout()
plt.show()
```

Algunas de las funciones mostradas:

```
fig,ax = plt.subplots(1,2,figsize=(14 , 6))
```

- **plt.subplots:** permite identificar la figura y el gráfico en sí, para poder personalizar. cada uno. Podríamos tener varios gráficos en una misma figura y personalizar cada uno por separado.
 - variable **fig**: la figura que contiene ambos gráficos.
 - **1** fila
 - **2** columnas
 - **figsize:** tamaño en pulgadas, 14 de ancho y 6 de alto.
 - variable **ax**: un arreglo con la cantidad de gráficos especificados, ubicados en posición 0 y posición 1.
- **plt.get_cmap:** define una paleta de colores específica.
- **colors:** crea una lista de colores, donde cada color se obtiene de la paleta en función de su índice.
- **labels:** las etiquetas para cada porción del gráfico de torta(pie).
- **labeldistance:** ubica las etiquetas a una distancia específica desde el centro del círculo.
- **tick_label:** las etiquetas para cada barra del gráfico (bar).
- **fig.tight_layout():** ajusta los gráficos al tamaño de la figura.

Análisis y gráficos en Streamlit

Podemos generar diferentes tipos de gráficos en [Streamlit](#):

- gráficos simples: le pasamos los datos directamente
 - **st.bar_chart()**

- **st.line_chart()**
- otros
- gráficos a través de librerías específicas: generamos la figura con la librería y luego la mostramos con las funciones de cada librería en Streamlit:
 - Matplotlib: **st.pyplot(figura)**
 - Plotly: **st.plotly_chart(figura)**
 - muchas más.
 - En el caso de usar las librerías específicas para graficar, la generación es igual que fuera de Streamlit, lo que necesitamos para verla es la función propia para cada librería.

Interacción con widgets

Los datos o gráficos que mostramos pueden generarse en función de elecciones que haga el usuario, algunos de los widgets que nos permiten esta interacción son:

- **st.multiselect**: da la opción de elegir varios valores de una lista
`st.multiselect('Título', [datos])`
- **st.selectbox**: se puede seleccionar una sola opción
`st.selectbox('Título', [datos])`
- **st.select_slider**: valores únicos
`st.select_slider('Título', options=opciones)`
- **st.slider** rango de valores:(minimo, maximo) es un tupla que representa los valores seleccionados
`st.slider('Rango de Años', minimo, maximo, (minimo, maximo))`

Gráficos en Streamlit

Matplotlib

- Generar el gráfico y mostrar usando st.write(figura)

Streamlit:

- **st.line_chart(data)**: crea un gráfico de líneas. data puede ser un DataFrame de pandas o una Series de pandas.
- **st.bar_chart(data)**: crea un gráfico de barras. Similar a st.line_chart(), data puede ser un DataFrame o una Series.
- **st.scatter_chart(data)**: crea un gráfico de dispersión. data debe ser un DataFrame con al menos dos columnas.
- [Y más.](#)

Seguimos la próxima ...